TY AIDS · G DIV · PROJECT 1

VelocityLLM

SLA-aware dynamic batching for LLM serving

Dynamic Batching Engine for LLM Inference Serving

THE PROBLEM

A fixed batch is a guess.

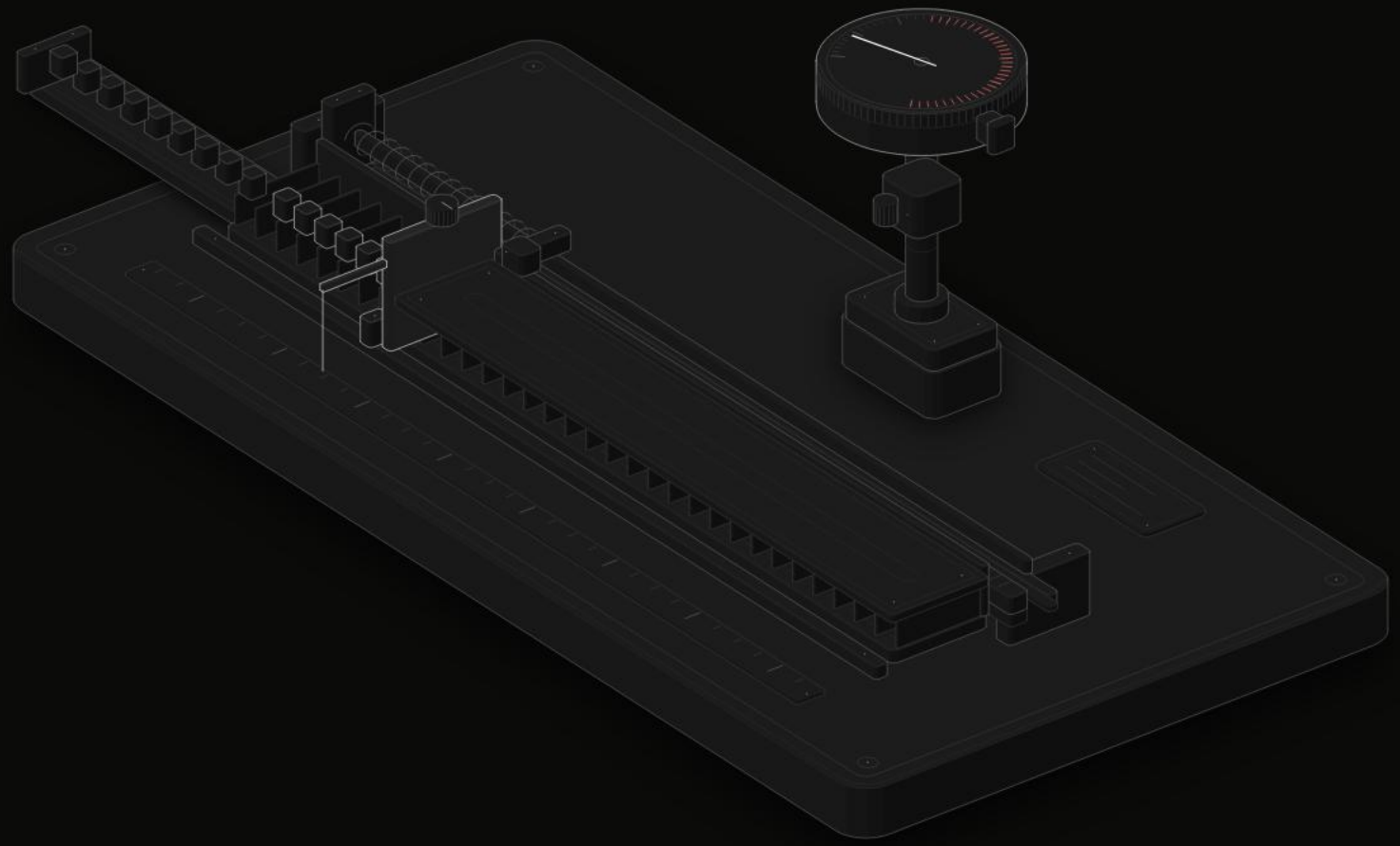
71.3 s

p99 latency with a static batch of 8 at 24 requests per second

The SLA is 8 s. Only 12% of requests made it.

Let the latency target set the batch.

01	**Measure**	p99 latency and GPU memory, continuously.
02	**Grow**	Open more slots while requests wait and p99 has room.
03	**Cut**	Pull back the moment either limit is hit.



Shutter

sets how many of the 32 slots are open

Dial

p99 latency against the red SLA zone

Spring

pulls the shutter back when p99 crosses

THE CONTROLLER

Additive up, multiplicative down.

GROW

+1, +2, +4

while requests wait and memory has room

CUT

×0.75

when p99 passes the SLA or memory hits 94%

RANGE

2 – 32

in flight, start at 8, step every 0.25 s

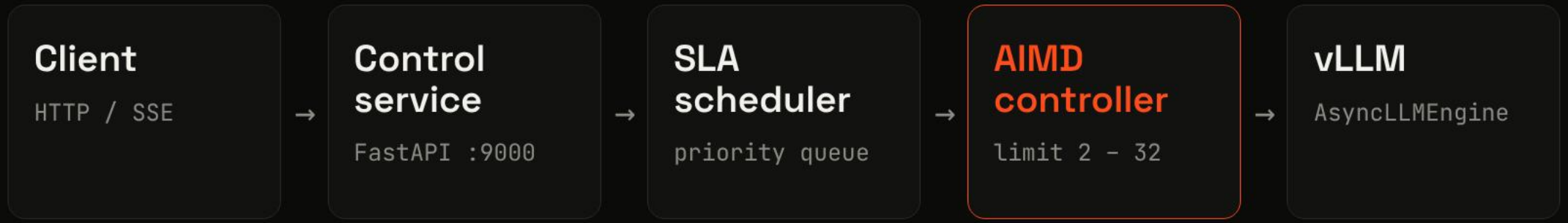
SOFT LIMIT

-1

when memory passes 88%, ahead of a cut

UNDER THE HOOD

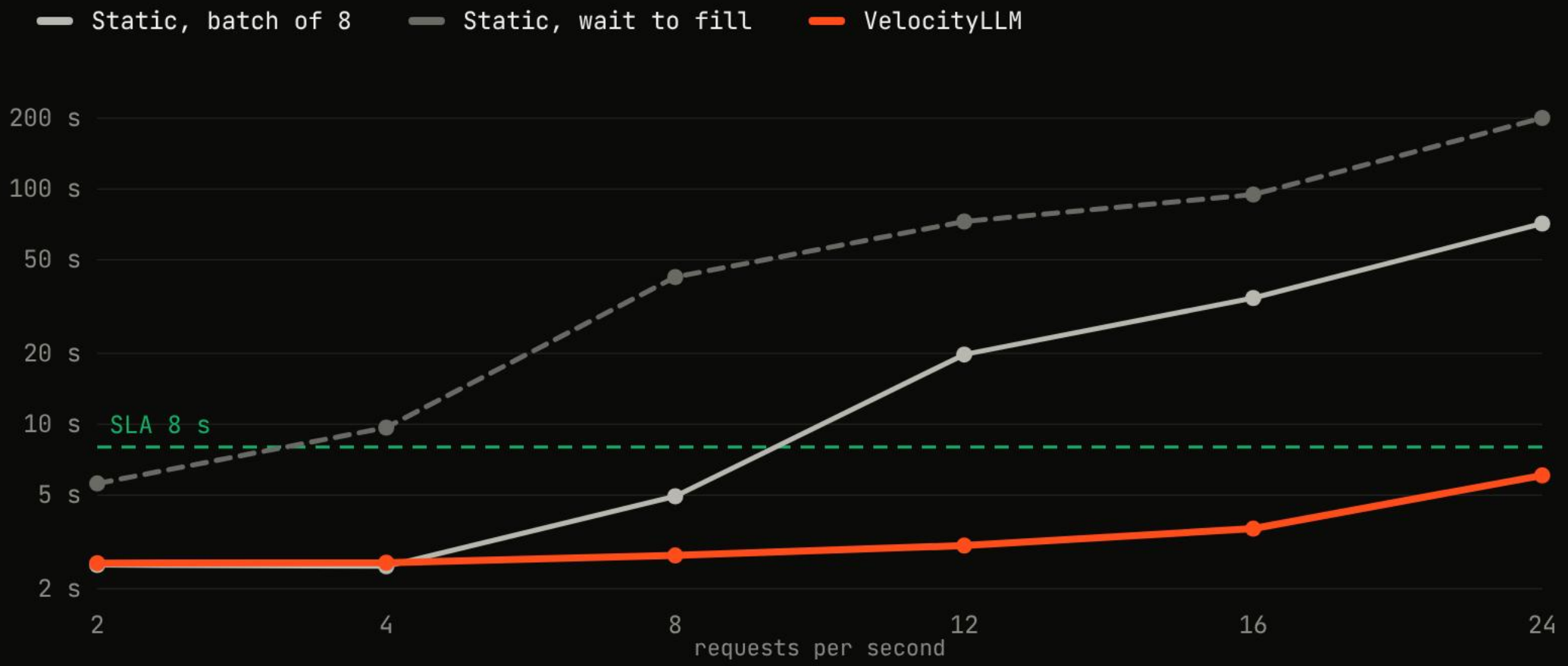
Drop-in in front of vLLM.



p99 and KV-cache memory flow back into the controller.
A mock backend and an HF runner cover CPU testing.

MEASURED · RTX 5050 8 GB · LLAMA-3.2-1B · 30 S PER LEVEL

p99 latency against load



AT 24 REQUESTS PER SECOND

Same GPU. Different outcome.

ON TIME

12% → 100%

static → VelocityLLM

GOODPUT, TOKENS/S

80 → 2,109

static → VelocityLLM

WHAT WE BUILT

More than a scheduler.

Engine

vLLM, HF Transformers and mock backends behind one interface

Control service

FastAPI endpoints, live metrics and controller state

Load tools

Async load generator and rate sweeps with SLA reports

Web app

A site that races static against dynamic, live

Package

pip install velocityllm on PyPI

Deploy

Docker image on vllm-openai and Triton scripts

12% on time became 100%.

pip install velocityllm

github.com/kartikpagariya25/VelocityLLM

KARTIK · VIKRANT · ADITYA · PRANALI